

Honorables Player System Development Framework

Canonical Source Specification

Version 3.0 (Revised)

Contents

1	Purpose	3
2	Scope	3
3	Design Objectives	3
4	Relationship Between Systems	4
4.1	Puffish Skills	4
4.2	Questing	4
4.3	GameStages	4
4.4	KubeJS	4
4.5	Equipment	4
5	Progression Architecture	4
5.1	Puffish Skills	5
5.2	Questing	5
5.3	GameStages	5
5.4	KubeJS	5
5.5	Traits	6
5.6	Attributes	6
5.7	Augments	6
5.8	Equipment	6
5.9	Interaction Between Systems	6
5.10	Separation of Responsibilities	6
6	Standard Class Tree Structure	7
6.1	Foundation	7
6.2	Root Node	7
6.3	Foundation Branches	8
6.4	Foundation Progression	8
6.5	Milestone Abilities	8
6.6	Subclass Unlock	8
6.7	Specialization	8
6.8	Mastery	8
6.9	Progression Flow	9
6.10	Design Goals	9
6.11	Implementation Notes	9
7	Specialization and Investment	9
7.1	Foundation vs. Specialization	9
7.2	Investment Philosophy	10
7.3	Soft Divergence	10
7.4	Crossroad Nodes	10
7.5	Internal Specialization	11
7.6	Ability Augments	11
7.7	Active Ability Philosophy	11
7.8	Composite Nodes	11
7.9	Decision Density	11
7.10	Breadth and Depth	12
7.11	Design Guidelines	12
8	Class Framework	12
8.1	Core Gameplay Loop	12
8.2	Economic Contribution	12
8.3	Primary Concerned Traits	13
8.4	Foundation Branches	13

8.5	Subclasses	13
8.6	Mastery	13
8.7	Canonical Class List	13
8.8	Miner	13
8.9	Naturalist	14
8.10	Adventurer	14
8.11	Warrior	15
8.12	Mage	15
8.13	Class Consistency	16
9	Skill Node Design	16
9.1	Node Composition	16
9.2	Attributes	16
9.3	Traits	16
9.4	Trait Derivation	17
9.5	Passive Effects	17
9.6	Active Abilities	17
9.7	Ability Augments	17
9.8	Commands	18
9.9	GameStages	18
9.10	Items	18
9.11	Recipes	18
9.12	Utility Unlocks	18
9.13	Composite Milestones	18
9.14	Ability Scaling	18
9.15	Node Density	19
9.16	Design Guidelines	19
10	Ability Design	19
10.1	Purpose of Active Abilities	19
10.2	Foundation Abilities	19
10.3	Subclass Abilities	20
10.4	Ability Count	20
10.5	Ability Scaling	20
10.6	Ability Augments	20
10.7	Ability Synergy	20
10.8	Quested Abilities	21
10.9	Ability Replacement	21
10.10	Active vs Passive Balance	21
10.11	Subclass Identity	21
10.12	Design Guidelines	21
11	Mastery Progression	21
11.1	Purpose	22
11.2	Philosophy	22
11.3	Progression Style	22
11.4	Capstone Design	22
11.5	Refinement over Expansion	22
11.6	Trait Integration	23
11.7	Equipment Interaction	23
11.8	Crossroads in Mastery	23
11.9	Endgame Investment	23
11.10	Replayability	23
11.11	Design Guidelines	23
12	Traits	24
12.1	Canonical Trait List	24
12.2	Purpose	24
12.3	Trait Calculation	24
12.4	Trait Recalculation	24
12.5	Permanent and Temporary Values	25
12.6	Relationship to Attributes	25
12.7	Relationship to Classes	25
12.8	Interclass Consistency	25
12.9	Design Philosophy	25
12.10	Future Expansion	26
12.11	Design Guidelines	26
13	Implementation Guidelines	26

13.1	Puffish Skills Responsibilities	26
13.2	KubeJS Responsibilities	26
13.3	GameStage Responsibilities	26
13.4	Quest Responsibilities	27
13.5	Equipment Responsibilities	27
13.6	Separation of Concerns	27
13.7	Data-Driven Design	27
13.8	Runtime Flexibility	27
13.9	Future-Proofing	28
13.10	Design Principles	28
14	Class Design Principles	28
14.1	Core Gameplay Loop	28
14.2	Economic Contribution	28
14.3	Distinct Identity	29
14.4	Foundation Consistency	29
14.5	Subclass Philosophy	29
14.6	Mastery Philosophy	29
14.7	Traits and Classes	29
14.8	Player Freedom	29
14.9	Long-Term Scalability	29
14.10	Design Guidelines	30

1 Purpose

This document serves as the canonical specification for the Honorables player system and progression framework. It defines the structure, responsibilities, and implementation guidelines for player progression using Puffish Skills, GameStages, KubeJS, and supporting gameplay systems.

The framework is intended to provide a stable reference for future player-system creation and maintenance. It should prioritize consistency, scalability, and long-term maintainability over individual implementation details.

Whenever possible, systems should be generalized so that new classes, subclasses, or mechanics can be introduced without requiring modifications to the underlying framework.

2 Scope

This framework defines:

- Class progression architecture
- Skill tree structure
- Foundation and specialization progression
- Mastery progression
- Trait and attribute interactions
- Active and passive ability placement
- Crossroad and specialization mechanics
- Canonical class definitions
- Reward definitions
- KubeJS and GameStage integration
- Implementation guidelines

This framework does **not** define:

- Individual skill node implementations
- Quest design
- Enemy balance
- Numerical balance values
- Loot tables
- Crafting recipes
- Individual class content beyond architectural requirements

Those systems should instead reference this framework while remaining independently maintainable.

3 Design Objectives

The Honorables progression framework is designed around several core technical objectives:

- Establish clear player identity through long-term progression.
- Encourage experimentation before specialization.
- Reward commitment without permanently restricting content.
- Keep the skill tree readable while supporting deep customization.
- Allow abilities, equipment, augments, and traits to scale together naturally.
- Separate progression structure from gameplay implementation.

- Minimize future maintenance costs through standardized architecture.
-

4 Relationship Between Systems

Player progression is distributed across multiple independent systems.

4.1 Puffish Skills

Defines:

- Class identity
- Progression structure
- Passive growth
- Active abilities
- Long-term specialization
- GameStage acquisition

Puffish Skills provides the structural backbone of progression but should not attempt to implement every gameplay mechanic.

4.2 Questing

Provides:

- Exploration rewards
- New gameplay opportunities
- Ability acquisition
- World interaction
- Progression outside of the class tree

Quest progression complements the skill tree rather than replacing it.

4.3 GameStages

Acts as the progression state manager.

GameStages determine access to:

- Recipes
- Structures
- Abilities
- Mechanics
- Progression milestones

They should remain binary unlock conditions whenever possible.

4.4 KubeJS

Acts as the runtime implementation layer.

Responsibilities include:

- Trait recalculation
- Derived attributes
- Temporary effects
- Ability behavior
- Scaling calculations
- Conditional logic
- Persistent player data

KubeJS should remain the authoritative source for gameplay calculations.

4.5 Equipment

Equipment provides an additional progression vector independent of class selection.

Equipment quality, enchantments, augments, and material quality should naturally interact with player traits and specialization without replacing them.

5 Progression Architecture

Honorables uses multiple complementary progression systems to define player identity and gameplay. Each system has a clearly defined responsibility and should avoid overlapping with the responsibilities of other systems whenever possible.

No individual system should attempt to encapsulate every aspect of progression. Instead, each component should contribute a specific layer that combines with the others to produce the complete player experience.

5.1 Puffish Skills

Puffish Skills provides the structural backbone of class progression.

Its responsibilities include:

- Defining class identity
- Organizing long-term progression
- Providing passive progression
- Providing active abilities
- Unlocking GameStages
- Granting attributes and derived bonuses
- Guiding specialization

Puffish Skills should establish **who the player is becoming**, rather than implementing every gameplay mechanic directly.

Nodes may grant any combination of:

- Attributes
- Passive effects
- Active abilities
- Ability augments
- GameStages
- Commands
- Items
- Recipes
- Utility unlocks

Node composition should remain flexible to allow meaningful progression milestones without artificially increasing tree size.

5.2 Questing

Questing serves as an independent progression system that complements class progression.

Quest rewards may include:

- New abilities
- Equipment
- Recipes
- Resources
- GameStages
- Permanent unlocks
- Story progression
- Exploration rewards

Questing should provide meaningful gameplay opportunities without replacing the role of the class tree.

Players should continue to find valuable progression outside of Puffish Skills throughout all stages of the game.

5.3 GameStages

GameStages represent binary progression states.

They should be used to gate access to:

- Recipes
- Structures
- Mechanics
- Quests
- Abilities
- Equipment
- Progression milestones

Whenever possible, GameStages should indicate whether a player has reached a particular milestone rather than storing numerical progression.

5.4 KubeJS

KubeJS functions as the runtime implementation layer for gameplay systems.

Responsibilities include:

- Trait recalculation
- Derived attribute calculation
- Temporary effects
- Cooldown management
- Ability behavior

- Conditional mechanics
- Equipment interaction
- Persistent player data
- Runtime progression logic

Complex gameplay behavior should generally be implemented through KubeJS rather than Puffish Skills.

5.5 Traits

Traits represent persistent character statistics that define long-term player growth.

Traits should be recalculated whenever progression changes and should provide a common scaling foundation for multiple gameplay systems.

Abilities, equipment, passives, and augments should naturally derive power from trait values rather than relying exclusively on ability-specific upgrades.

5.6 Attributes

Attributes provide localized progression within the broader trait framework.

Attribute investment should reinforce specialization while remaining useful across multiple mechanics.

Improving a character should naturally improve many aspects of gameplay simultaneously.

5.7 Augments

Augments modify or extend existing mechanics.

Rather than introducing entirely new gameplay systems, augments should primarily:

- Expand existing abilities
- Alter behavior
- Introduce conditional interactions
- Improve specialization
- Create build diversity

Subclass progression should increasingly rely on augments as investment deepens.

5.8 Equipment

Equipment exists as an independent progression vector.

Equipment progression may include:

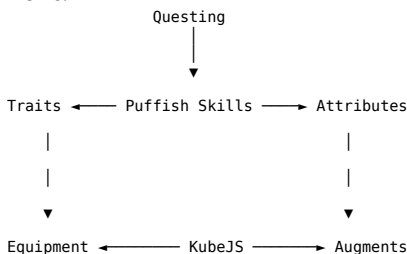
- Material quality
- Craftsmanship quality
- Enchantments
- Perks
- Augments
- Durability
- Affixes

Equipment should complement player progression rather than replace it.

Character identity should continue to originate primarily from long-term investment into progression systems.

5.9 Interaction Between Systems

The complete player experience emerges through the interaction of multiple systems rather than any individual component.



Each system should remain independently maintainable while reinforcing the others through shared progression.

5.10 Separation of Responsibilities

Future development should preserve clear ownership between systems.

System	Primary Responsibility
Puffish Skills	Class identity and structural progression
Questing	World progression and external rewards
GameStages	Binary progression state
KubeJS	Runtime implementation and calculations
Traits	Long-term character growth
Attributes	Specialized player statistics
Augments	Build refinement and mechanic modification
Equipment	External progression and itemization

Maintaining this separation improves scalability, reduces implementation complexity, and minimizes overlap between progression systems.

6 Standard Class Tree Structure

Every playable class in Honorables follows a standardized progression architecture. This structure exists to provide consistency between classes while allowing each specialization to express its own gameplay loop and identity.

The overall progression of a class is divided into three major phases:

- Foundation
- Specialization
- Mastery

Each phase serves a distinct purpose and should be designed with different priorities.

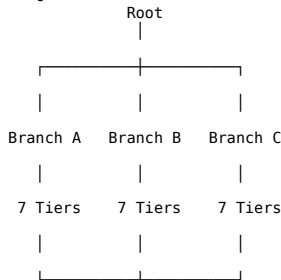
6.1 Foundation

The Foundation represents the player's introduction to a class.

Its purpose is to:

- Establish class identity.
- Teach the core gameplay loop.
- Encourage experimentation.
- Build general competence.
- Delay major specialization decisions.

Every class should share the same foundational structure.



This results in:

- 1 Root node
- 3 Branches
- 7 Tiers per branch
- 21 Foundation nodes
- 22 total nodes including the Root

This structure should remain consistent across every class unless explicitly justified otherwise.

6.2 Root Node

Every class begins with a single Root node.

The Root exists to establish the player's initial identity and provide access to the remainder of the progression tree.

Typical Root rewards may include:

- Starter attributes
- Passive bonuses
- Introductory active ability
- Commands
- GameStages
- Tutorial progression

The Root should introduce the class without immediately forcing specialization.

6.3 Foundation Branches

The three Foundation branches represent broad aspects of the class rather than strict specializations. They should:

- Teach different facets of the gameplay loop.
- Encourage experimentation.
- Remain generally useful regardless of future subclass selection.

Investments made during Foundation progression should continue to provide value throughout the entire game. Branches should avoid introducing mechanics that immediately define an irreversible playstyle.

6.4 Foundation Progression

Foundation progression should remain relatively low in decision density.

Players should primarily focus on:

- Learning mechanics.
- Building attributes.
- Unlocking passive growth.
- Sampling class identity.

Major specialization should be intentionally delayed until subclass unlock.

6.5 Milestone Abilities

Foundation branches may contain milestone abilities that preview future specialization.

These abilities should:

- Demonstrate branch identity.
- Remain generally useful.
- Avoid forcing subclass commitment.
- Continue scaling throughout progression.

Milestone abilities should complement experimentation rather than define the player's final build.

6.6 Subclass Unlock

After completing Foundation progression, players gain access to subclass specialization.

Subclass unlock represents the transition from exploration to commitment.

At this point:

- Decision density increases.
- Investment becomes more meaningful.
- Additional mechanics become available.
- Identity becomes increasingly specialized.

Subclass unlock should expand progression possibilities rather than restrict them.

6.7 Specialization

Subclass progression exists to refine an existing identity rather than replace it.

Players should continue developing the gameplay loop introduced during Foundation while gaining increasingly specialized tools and interactions.

Specialization should prioritize:

- Passive synergies
- Attributes
- Augments
- Crossroads
- Ability refinement
- High-impact decisions

The goal is to deepen mastery without overwhelming players with unnecessary complexity.

6.8 Mastery

Mastery represents the culmination of long-term investment.

It should reward dedication through refinement rather than breadth.

Mastery rewards may include:

- Signature passives
- Capstone mechanics
- Powerful augments

- Transformative abilities
- Endgame specialization bonuses

Mastery should enhance an established identity rather than fundamentally redefine it.

6.9 Progression Flow

The complete progression of every class should generally follow the pattern below.

```

Root
↓
Foundation
↓
Experimentation
↓
Subclass Unlock
↓
Commitment
↓
Specialization
↓
Refinement
↓
Mastery

```

This progression intentionally shifts from broad exploration toward increasingly deliberate investment.

6.10 Design Goals

The standardized class structure exists to ensure:

- Consistent onboarding across classes.
- Predictable progression pacing.
- Meaningful specialization.
- Long-term replayability.
- Future scalability.

Individual classes should express uniqueness through gameplay mechanics and specialization rather than structural differences.

6.11 Implementation Notes

When designing future classes:

- Preserve the standardized Foundation structure.
- Use subclass progression to increase decision density.
- Avoid front-loading specialization.
- Encourage experimentation before commitment.
- Ensure Foundation investments remain valuable after specialization.
- Treat Mastery as refinement rather than expansion.

The progression architecture should remain recognizable regardless of class while allowing each specialization to develop its own identity through continued investment.

7 Specialization and Investment

Honorables emphasizes specialization through continued investment rather than permanent restriction. Players should be encouraged to develop unique identities over time while preserving the ability to diversify through additional commitment.

The progression framework should reward focus without relying on irreversible decisions or mutually exclusive paths.

7.1 Foundation vs. Specialization

The progression system is intentionally divided into two distinct phases.

7.1.1 Foundation

The Foundation phase prioritizes:

- Exploration
- Experimentation
- Learning class mechanics

- Building general competence

Players should be able to invest broadly with relatively little penalty.

The objective is to help players discover which aspects of the class they most enjoy before making significant commitments.

7.1.2 Specialization

Following subclass unlock, progression should shift toward refinement.

The primary goals become:

- Reinforcing identity
- Increasing decision density
- Encouraging long-term planning
- Supporting unique builds

Rather than introducing large numbers of unrelated mechanics, specialization should deepen existing gameplay systems.

7.2 Investment Philosophy

Specialization should emerge naturally from repeated investment.

The framework favors opportunity cost over exclusion.

Players should always be capable of pursuing additional progression paths, but doing so should require progressively greater investment.

This allows players to experiment freely while preserving the value of long-term commitment.

7.3 Soft Divergence

Soft Divergence is the primary mechanism used to encourage specialization.

Whenever multiple progression options are available, selecting one should increase the investment required to pursue the alternatives rather than permanently locking them.

Example:

Choice A

Choice B

Choice C

After investing into Choice A:

Choice A Normal Cost

Choice B Increased Cost

Choice C Increased Cost

After later investing into Choice B:

Choice A Normal Cost

Choice B Increased Cost

Choice C Further Increased Cost

The player remains free to acquire every option eventually, but meaningful specialization emerges through accumulated opportunity cost.

7.4 Crossroad Nodes

Crossroad Nodes represent deliberate branching points within progression.

Unlike traditional branching skill trees, Crossroads do not permanently separate players into incompatible builds.

Instead, they encourage prioritization.

A Crossroad may contain any combination of:

- Attributes
- Passive effects
- Active abilities
- Ability augments
- Utility unlocks
- Commands
- GameStages
- Recipes
- Items

Each option represents a different investment direction rather than an exclusive path.

7.5 Internal Specialization

Soft Divergence applies not only between subclasses but also within them.

For example, a Spellsword may encounter a Crossroad offering:

- Arcane Blade
- Spell Echo
- Mana Efficiency

Investing into one should naturally encourage continued refinement while preserving eventual access to the others. The resulting player identity emerges from cumulative investment patterns rather than hard restrictions.

7.6 Ability Augments

Subclass progression should increasingly favor augmenting existing mechanics over introducing new ones.

Augments may:

- Alter behavior
- Introduce conditional effects
- Expand utility
- Improve synergy
- Reinforce specialization

This allows a relatively small number of active abilities to support a wide variety of builds.

7.7 Active Ability Philosophy

Active abilities should remain meaningful throughout progression.

Rather than repeatedly introducing additional buttons, specialization should frequently improve existing mechanics through:

- Trait scaling
- Attribute investment
- Passives
- Augments
- Equipment interactions

Players should become more effective through mastery rather than hotbar expansion.

7.8 Composite Nodes

Progression nodes are not limited to a single reward type.

A single node may simultaneously grant:

- Attributes
- Passive bonuses
- Active abilities
- Ability augments
- GameStages
- Commands
- Items
- Recipes
- Utility unlocks

Node design should prioritize meaningful progression milestones rather than artificially separating related rewards across multiple nodes.

7.9 Decision Density

Decision density should intentionally increase after subclass unlock.

Foundation progression should remain straightforward.

Specialization should instead encourage players to weigh multiple valuable investments against one another.

Increasing decision density does not necessarily require increasing node count.

Meaningful choices may instead emerge through:

- Crossroads
 - Composite rewards
 - Escalating investment costs
 - Augment interactions
 - Passive synergies
-

7.10 Breadth and Depth

Progression should naturally transition from breadth toward depth.

Early progression should encourage players to explore multiple areas.

Later progression should encourage players to refine an established identity.

A generalized progression model is:

Foundation

↓

Experimentation

↓

Commitment

↓

Refinement

↓

Mastery

At no point should the player be permanently prevented from pursuing additional investments, but each additional specialization should represent an increasingly significant commitment.

7.11 Design Guidelines

When designing subclass progression:

- Prefer refinement over expansion.
- Prefer opportunity cost over restriction.
- Favor augments over excessive active abilities.
- Increase decision density after specialization.
- Encourage identity through repeated investment.
- Preserve long-term flexibility while rewarding commitment.

The objective of specialization is not to remove player choice, but to make every investment increasingly meaningful.

8 Class Framework

Every class within Honorables should represent a distinct gameplay loop and long-term role within the world. Classes are intended to define player identity through progression and investment rather than through weapon restrictions or arbitrary limitations.

The purpose of a class is to answer the question:

“How does this player naturally contribute to the world?”

Classes should therefore be designed around gameplay responsibilities rather than isolated mechanics.

8.1 Core Gameplay Loop

Every class definition should begin by identifying its Core Gameplay Loop.

The Core Gameplay Loop describes the primary activities through which the class progresses and interacts with the game world.

Examples include:

- Resource acquisition
- Resource production
- Combat mastery
- Exploration
- Magical specialization

The Core Gameplay Loop should remain recognizable throughout every stage of progression.

Subclasses should refine this loop rather than replace it.

8.2 Economic Contribution

Every class should provide a meaningful contribution to the broader player ecosystem.

Economic Contribution defines what value a class naturally creates for other players.

Examples include:

- Equipment production
- Food production
- Enchantments
- Exploration
- Infrastructure

- Combat capability

A class should remain valuable even when viewed independently of combat performance.

8.3 Primary Concerned Traits

Each class should naturally emphasize a subset of player traits.

These traits represent the statistics most commonly improved through progression and should reinforce the intended gameplay loop.

Primary traits should guide progression without restricting player freedom.

Secondary investments should always remain viable where appropriate.

8.4 Foundation Branches

The three Foundation branches should represent complementary aspects of the Core Gameplay Loop.

Branches should not function as hidden subclasses.

Instead, they should provide multiple perspectives on the same profession while remaining broadly applicable regardless of future specialization.

Each branch should teach different mechanics while continuing to support the overall identity of the class.

8.5 Subclasses

Subclasses exist to refine player identity after Foundation progression.

A subclass should:

- Reinforce the Core Gameplay Loop.
- Expand specialization.
- Increase decision density.
- Introduce unique mechanics.
- Encourage long-term commitment.

Subclasses should not invalidate Foundation investments.

Instead, they should build naturally upon them.

8.6 Mastery

Mastery represents the culmination of repeated investment into a specialization.

Mastery should reward expertise rather than breadth.

Capstone rewards should:

- Reinforce existing mechanics.
- Deepen specialization.
- Improve established gameplay loops.
- Avoid introducing unrelated systems.

Mastery should feel like the perfection of an existing discipline.

8.7 Canonical Class List

Honorables currently defines five primary classes.

- Miner
- Naturalist
- Adventurer
- Warrior
- Mage

Each serves a distinct gameplay role while supporting interaction with the larger progression ecosystem.

8.8 Miner

8.8.1 Core Gameplay Loop

Acquire, refine, and transform raw resources into tools, equipment, and industrial infrastructure.

8.8.2 Economic Contribution

- Raw materials
- Equipment
- Infrastructure
- Industrial production

8.8.3 Primary Concerned Traits

- Strength

- Intelligence
- Constitution

8.8.4 Subclasses

8.8.4.1 Quarryman Specializes in extraction and resource acquisition.

8.8.4.2 Blacksmith Specializes in forging, equipment quality, and craftsmanship.

8.8.4.3 Engineer Specializes in industrial systems, automation, and infrastructure.

Ore discovery should be considered part of base Miner progression rather than subclass identity.

8.9 Naturalist

8.9.1 Core Gameplay Loop

Produce renewable resources through cultivation, animal husbandry, and biological processing.

8.9.2 Economic Contribution

- Food
- Renewable materials
- Consumables
- Potions
- Poisons

8.9.3 Primary Concerned Traits

- Intelligence
- Constitution
- Dexterity

8.9.4 Subclasses

8.9.4.1 Farmer Specializes in crop cultivation and plant production.

8.9.4.2 Rancher Specializes in:

- Livestock
- Fishing
- Aquaculture
- Bees
- Renewable animal resources

8.9.4.3 Brewer Specializes in:

- Potions
 - Poisons
 - Fermentation
 - Food processing
 - Consumable production
-

8.10 Adventurer

8.10.1 Core Gameplay Loop

Discover, explore, and unlock opportunities throughout the world.

8.10.2 Economic Contribution

- Exploration
- Discovery
- Mapping
- Dungeon progression
- Relic acquisition

8.10.3 Primary Concerned Traits

- Constitution
- Dexterity
- Intelligence

8.10.4 Subclasses

8.10.4.1 Pathfinder Focuses on traversal and mobility.

8.10.4.2 Treasure Hunter Focuses on structures, ruins, and valuable discoveries.

8.10.4.3 Cartographer Focuses on mapping, surveying, and navigation.

Each specialization should encourage interaction with unexplored content while supporting the overall exploration loop.

8.11 Warrior**8.11.1 Core Gameplay Loop**

Master physical combat through discipline, training, and martial specialization.

8.11.2 Economic Contribution

- Combat capability
- Group defense
- Boss encounters
- Frontline engagement

8.11.3 Primary Concerned Traits

- Strength
- Constitution
- Dexterity

8.11.4 Subclasses**8.11.4.1 Knight** Primary emphasis:

- Defense
- Shields
- Survivability
- Formation combat

8.11.4.2 Berserker Primary emphasis:

- Raw offense
- Heavy weapons
- Sustained melee combat

8.11.4.3 Skirmisher Primary emphasis:

- Dexterity
- Critical strikes
- Precision
- Mobility
- Finesse weapons

Skirmisher replaces the traditional Ranger role by specializing in agile martial combat rather than ranged weapon exclusivity.

8.12 Mage**8.12.1 Core Gameplay Loop**

Manipulate magical systems to influence combat, equipment, and progression.

8.12.2 Economic Contribution

- Spellcasting
- Equipment enhancement
- Magical support
- Arcane specialization

8.12.3 Primary Concerned Traits

- Intelligence

8.12.4 Subclasses**8.12.4.1 Elementalist** Focuses on direct magical offense.**8.12.4.2 Spellsword** Blends martial combat with arcane techniques.**8.12.4.3 Enchanter** Focuses on:

- Weapon imbue ment
- Armor imbue ment
- Magical perks
- Equipment enhancement
- Arcane support

The Enchanter serves as one of the primary providers of long-term equipment progression within the game economy.

8.13 Class Consistency

Every future class should define:

- A Core Gameplay Loop
- An Economic Contribution
- Primary Concerned Traits
- Three Foundation branches
- Three Subclasses
- A coherent Mastery path

The purpose of this standardization is not to reduce creativity, but to ensure every class remains equally understandable, expandable, and maintainable while contributing meaningfully to the larger progression framework.

9 Skill Node Design

Skill Nodes are the fundamental building blocks of progression within Puffish Skills. Every node should represent a meaningful investment that reinforces player identity, specialization, or long-term progression.

Nodes should not be artificially constrained to a single reward type. Instead, each node should be viewed as a progression milestone capable of advancing multiple aspects of a character simultaneously.

The objective is to maximize meaningful decisions while minimizing unnecessary tree complexity.

9.1 Node Composition

Every node may grant any combination of:

- Attributes
- Passive effects
- Active abilities
- Ability augments
- Commands
- GameStages
- Items
- Recipes
- Utility unlocks

A node should package together logically related rewards whenever possible.

For example, a Blacksmith milestone may simultaneously:

- Increase attack damage.
- Improve forging quality.
- Unlock a recipe.
- Grant a GameStage.

This should be considered one progression event rather than four separate nodes.

9.2 Attributes

Attributes are the primary numerical rewards granted directly by Puffish Skills.

These include both vanilla and modded attributes and should be used to provide measurable improvements to the player.

Examples include:

- Maximum Health
- Armor
- Armor Toughness
- Attack Damage
- Attack Speed
- Movement Speed
- Mana
- Spell Power
- Critical Chance
- Critical Damage

Attributes should reinforce the intended gameplay loop of the surrounding branch or specialization.

9.3 Traits

Traits are abstract representations of long-term character capability.

The canonical traits are:

- Strength
- Endurance
- Constitution

- Dexterity
- Agility
- Intelligence
- Wisdom

Traits should **not** be granted directly by Puffish Skills.

Instead, KubeJS should derive trait values from the complete state of the player, including:

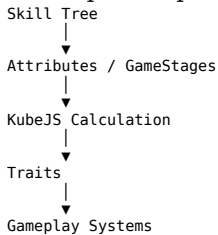
- Skill tree progression
- Attributes
- Equipment
- Active effects
- Persistent data
- Temporary modifiers

Traits exist as the common language through which gameplay systems communicate.

9.4 Trait Derivation

Traits should be recalculated whenever relevant progression changes occur.

The expected progression flow is:



This separation allows traits to respond dynamically to buffs, debuffs, equipment changes, and progression without requiring modifications to Puffish Skills.

9.5 Passive Effects

Passive effects modify gameplay without requiring direct player activation.

Examples include:

- Increased mining efficiency
- Reduced stamina consumption
- Increased crop yield
- Improved block chance
- Faster mana regeneration

Passives should generally reinforce specialization and create meaningful synergies rather than simply increasing statistics.

9.6 Active Abilities

Active abilities provide explicit player actions.

They should be introduced carefully and intentionally.

The Foundation section of the tree should contain relatively few active abilities.

Subclass progression may introduce additional abilities where appropriate, but should primarily focus on refining and augmenting existing mechanics.

Every active ability should remain useful throughout progression.

9.7 Ability Augments

Augments modify existing abilities instead of introducing entirely new mechanics.

Examples include:

- Additional effects
- New conditions
- Resource refunds
- Improved scaling
- Alternative behavior
- Conditional interactions

Augments should become increasingly common after specialization.

A well-designed augment system allows a small number of active abilities to support many distinct builds.

9.8 Commands

Nodes may execute commands during acquisition.

Typical uses include:

- Initialization
- Setup
- Item distribution
- Synchronization
- Progression migration

Commands should avoid implementing gameplay logic that belongs in KubeJS.

9.9 GameStages

GameStages represent permanent progression milestones.

Nodes may grant GameStages to unlock:

- Recipes
- Features
- Quests
- Structures
- Mechanics
- Abilities
- Progression tiers

GameStages should remain binary whenever practical.

9.10 Items

Nodes may grant physical items.

Examples include:

- Starter equipment
- Progression tokens
- Keys
- Quest items
- Class-specific tools

Item rewards should complement progression rather than replace it.

9.11 Recipes

Recipes may be unlocked directly or through associated GameStages.

Recipe progression should encourage specialization and reinforce the economic role of the class.

9.12 Utility Unlocks

Nodes may provide non-combat progression such as:

- Travel improvements
- Interface enhancements
- Quality-of-life features
- Convenience mechanics
- Crafting utilities

Utility rewards should support gameplay without overshadowing specialization.

9.13 Composite Milestones

Multiple related rewards should frequently be grouped into a single node.

Example:

```
Master Smith
+Attack Damage
+Forging Quality
Unlock Tempering
Grant Blacksmith Tier II
```

The objective is to create satisfying progression milestones rather than inflate node count.

9.14 Ability Scaling

Abilities should scale naturally from the overall state of the character.

Scaling sources may include:

- Traits
- Attributes
- Equipment quality
- Enchantments
- Passive bonuses
- Augments
- Temporary effects

Improving the character should improve every owned ability where appropriate.

Avoid excessive chains of isolated ability-specific upgrades.

9.15 Node Density

The complexity of the tree should come from decision making rather than quantity.

Prefer:

- Composite rewards
- Crossroads
- Augments
- Synergies
- Opportunity cost

over excessive numbers of isolated nodes.

A smaller tree with meaningful decisions is preferable to a larger tree filled with incremental upgrades.

9.16 Design Guidelines

When designing nodes:

- Reinforce class identity.
- Support the surrounding gameplay loop.
- Bundle related rewards together.
- Favor refinement over expansion.
- Use augments to deepen mechanics.
- Preserve long-term value.
- Encourage meaningful investment.

Every node should feel like a deliberate step toward mastery rather than a mandatory statistical checkpoint.

10 Ability Design

Active abilities represent explicit actions available to the player and should reinforce the identity established by the surrounding progression tree.

Unlike attributes or passive effects, active abilities introduce mechanical complexity and increase the player's cognitive load. As a result, abilities should be introduced deliberately and remain meaningful throughout the entirety of progression.

The objective is to create a small collection of impactful tools rather than an ever-expanding hotbar.

10.1 Purpose of Active Abilities

Active abilities exist to:

- Reinforce class identity.
- Express specialization.
- Introduce unique gameplay opportunities.
- Reward progression milestones.
- Differentiate player builds.

Abilities should solve problems in interesting ways rather than simply increasing damage or statistics.

10.2 Foundation Abilities

The Foundation phase should introduce only a limited number of active abilities.

Typically this consists of:

- One introductory Root ability.
- Approximately one milestone ability per Foundation branch.

These abilities should:

- Teach class mechanics.
- Preview future specialization.
- Remain useful after subclass unlock.

- Avoid prematurely defining a final playstyle.

Foundation progression should remain focused on learning rather than specialization.

10.3 Subclass Abilities

Subclass progression may introduce additional active abilities that significantly alter gameplay.

These abilities should:

- Reinforce subclass identity.
- Expand tactical options.
- Encourage long-term investment.
- Work naturally with existing mechanics.

Subclass abilities should feel transformative without invalidating previous progression.

10.4 Ability Count

The total number of active abilities available to a player should remain intentionally limited.

Depth should come from:

- Augments
- Traits
- Equipment
- Passives
- Scaling
- Synergy

rather than simply acquiring additional abilities.

Players should master existing tools instead of continuously replacing them.

10.5 Ability Scaling

Abilities should scale primarily through the growth of the character.

Scaling sources include:

- Traits
- Attributes
- Equipment quality
- Enchantments
- Passive bonuses
- Ability augments
- Temporary effects

Whenever possible, improvements to the player should naturally improve all relevant abilities.

Avoid isolated upgrade chains that only benefit a single ability.

10.6 Ability Augments

Augments modify existing abilities rather than creating new ones.

Common augment effects include:

- Additional projectiles
- Area expansion
- Resource refunds
- Conditional bonuses
- Alternative activation
- Status application
- Combo interactions
- Utility improvements

Augments should become increasingly common as specialization progresses.

10.7 Ability Synergy

Abilities should interact naturally with:

- Traits
- Passives
- Equipment
- Enchantments
- Other abilities
- Temporary effects

The effectiveness of an ability should depend on the complete state of the character rather than on isolated upgrades. Synergies encourage specialization while preserving flexibility.

10.8 Quested Abilities

Not every ability should originate from the skill tree.

Quest progression provides an opportunity to introduce:

- Utility mechanics
- Exploration tools
- Signature techniques
- World-specific interactions
- Optional gameplay systems

Quested abilities complement class progression while rewarding engagement with the world.

The skill tree should define the player's foundation, while questing expands their capabilities beyond it.

10.9 Ability Replacement

Abilities should rarely become obsolete.

Earlier abilities should continue to benefit from:

- Trait scaling
- Equipment upgrades
- New augments
- Passive synergies
- Mastery progression

Players should feel as though they are refining familiar tools rather than abandoning them.

10.10 Active vs Passive Balance

Whenever possible, progression should favor passive improvement over additional active mechanics.

Passive growth provides:

- Lower cognitive load
- Better scalability
- Broader gameplay impact
- Greater build flexibility

Active abilities should remain special moments of progression rather than routine rewards.

10.11 Subclass Identity

Subclasses should primarily distinguish themselves through:

- Unique abilities
- Ability augments
- Passive interactions
- Specialized scaling
- Mechanical synergies

A subclass should feel fundamentally different not because it possesses many more abilities, but because it uses its abilities differently.

10.12 Design Guidelines

When designing active abilities:

- Keep the total ability count manageable.
- Introduce new abilities deliberately.
- Prefer augments over redundant abilities.
- Ensure early abilities remain relevant.
- Scale through character growth rather than isolated upgrades.
- Reinforce the surrounding gameplay loop.

Every active ability should represent a defining moment in progression and remain valuable from acquisition through mastery.

11 Mastery Progression

Mastery represents the final stage of class progression and the culmination of long-term investment into a specialization. Unlike Foundation, which emphasizes experimentation, or Specialization, which emphasizes commitment, Mastery ex-

ists to perfect an established playstyle through refinement rather than expansion.

Mastery should reward dedication without fundamentally changing the identity that the player has developed throughout progression.

11.1 Purpose

The primary objectives of Mastery are to:

- Refine existing gameplay loops.
- Reward long-term commitment.
- Reinforce player identity.
- Enhance established mechanics.
- Provide meaningful endgame progression.

Mastery should never invalidate previous investments or redefine the specialization entirely.

11.2 Philosophy

Mastery is not intended to make the player broader.

Instead, it should make the player exceptionally proficient within their chosen discipline.

The distinction can be summarized as:

Foundation

Learn.

↓

Specialization

Commit.

↓

Mastery

Perfect.

Players should feel that they are becoming legendary practitioners of an already established craft rather than acquiring unrelated mechanics.

11.3 Progression Style

Mastery progression should favor refinement over expansion.

Typical rewards include:

- Signature passives
- Powerful augments
- Trait scaling improvements
- Conditional mechanics
- Specialized interactions
- Capstone abilities

Mastery should avoid introducing large numbers of entirely new systems.

11.4 Capstone Design

Every specialization should culminate in one or more capstone milestones.

Capstones should represent defining achievements that reinforce the specialization's identity.

Examples include:

- Master Blacksmith forging exceptional equipment.
- Enchanter imbuing multiple magical properties.
- Berserker entering an advanced combat state.
- Brewer creating otherwise impossible consumables.

Capstones should feel earned through sustained investment.

11.5 Refinement over Expansion

Mastery should primarily improve mechanics the player already understands.

Examples include:

- Existing abilities becoming more versatile.
- Existing passives gaining additional interactions.
- Existing augments unlocking new combinations.
- Existing gameplay loops becoming more efficient.

The player should recognize their own progression rather than needing to relearn their class.

11.6 Trait Integration

Mastery should increasingly leverage the underlying trait framework.

High-level mechanics may scale from:

- Strength
- Endurance
- Constitution
- Dexterity
- Agility
- Intelligence
- Wisdom

The greater the player's investment into these traits, the more effectively Mastery mechanics should perform.

11.7 Equipment Interaction

Mastery should naturally synergize with equipment progression.

Examples include:

- Improved quality scaling.
- Better enchantment utilization.
- Greater effectiveness from crafted gear.
- Increased efficiency from specialized tools.
- Enhanced interaction with equipment augments.

Equipment should complement Mastery rather than replace it.

11.8 Crossroads in Mastery

Mastery may continue to present meaningful choices through Crossroad Nodes.

These choices should encourage further refinement without permanently excluding alternatives.

For example:

Master Enchanter



Potency Efficiency Utility

Investing into one direction should increase the opportunity cost of pursuing the others while preserving eventual access.

11.9 Endgame Investment

Mastery progression should represent some of the most significant investments available to the player.

High-impact rewards should justify their cost through:

- Increased specialization
- Greater efficiency
- Enhanced synergy
- Expanded mastery of existing systems

Mastery should remain aspirational without becoming mandatory for ordinary gameplay.

11.10 Replayability

Mastery should encourage different players within the same specialization to develop distinct identities.

Two players following identical subclasses may still diverge through:

- Crossroad investment
- Augment selection
- Trait emphasis
- Equipment choices
- Passive synergies

The resulting diversity should emerge naturally from investment patterns rather than hard restrictions.

11.11 Design Guidelines

When designing Mastery progression:

- Refine existing mechanics before creating new ones.

- Reward dedication rather than breadth.
- Favor augments and passives over excessive active abilities.
- Integrate naturally with traits and equipment.
- Preserve subclass identity.
- Keep capstones impactful and memorable.

Mastery should represent the highest expression of a specialization, allowing players to become exceptional at what they have chosen rather than simply granting them access to more content.

12 Traits

Traits represent the universal capabilities of a character and form the common foundation upon which all progression systems interact.

Unlike attributes, which are granted directly through progression systems, traits are abstract values calculated from the player's complete state and interpreted by gameplay systems through KubeJS.

The trait framework exists to provide a consistent language for evaluating player capability while remaining independent of any individual class, specialization, or mechanic.

Traits should remain broad, stable, and reusable throughout the lifetime of the project.

12.1 Canonical Trait List

Honorables defines seven primary traits.

- Strength
- Endurance
- Constitution
- Dexterity
- Agility
- Intelligence
- Wisdom

These seven traits form the complete primary trait framework and should be reused whenever possible.

Additional primary traits should not be introduced without significant justification.

12.2 Purpose

Traits measure **capability**, not **specialization**.

They answer questions such as:

- How physically strong is this character?
- How agile is this character?
- How intelligent is this character?
- How resilient is this character?

Traits should not directly determine mastery within a profession or discipline.

Instead, they provide common inputs that allow different gameplay systems to evaluate the player in a consistent manner.

12.3 Trait Calculation

Traits are runtime values derived through KubeJS.

Possible inputs include:

- Skill tree progression
- Attributes
- Equipment
- Equipment quality
- GameStages
- Persistent player data
- Temporary effects
- Buffs and debuffs

The exact implementation is intentionally flexible and may evolve over time without requiring modifications to Puffish Skills.

12.4 Trait Recalculation

Trait values should be recalculated whenever relevant player state changes.

Typical recalculation events include:

- Skill tree progression

- Equipment changes
- Buff application
- Buff expiration
- Potion effects
- Poison effects
- Temporary modifiers
- Class progression updates

Centralizing recalculation ensures that all gameplay systems evaluate the player's current effective capability.

12.5 Permanent and Temporary Values

Trait systems should distinguish between permanent progression and temporary modification.

Permanent values represent long-term character development.

Temporary values represent transient gameplay effects such as:

- Potions
- Poisons
- Environmental hazards
- Equipment bonuses
- Status conditions

Temporary modifiers should influence effective trait values without overwriting permanent progression.

This allows gameplay systems to remain dynamic while preserving long-term investment.

12.6 Relationship to Attributes

Attributes and traits serve different purposes.

Attributes are direct numerical rewards granted by progression systems and consumed by Minecraft or installed mods.

Traits are abstract representations of player capability calculated from the complete state of the character.

Attributes should therefore contribute to trait calculation rather than replace the trait framework itself.

12.7 Relationship to Classes

Classes do not define traits.

Instead, classes define how traits are expressed.

Two players may possess identical trait values while producing dramatically different outcomes because of their specialization.

For example, a Farmer and a Warrior may each possess 100 Strength.

Both are physically capable of wielding a greatsword.

However, the Warrior's class progression provides specialized passives, perks, active abilities, augments, and combat mechanics that allow them to utilize the weapon far more effectively than the Farmer.

Likewise, a Blacksmith and an Enchanter may each possess 100 Intelligence.

The Blacksmith's progression makes them exceptionally skilled at forging equipment, while the Enchanter's progression makes them exceptionally skilled at imbuing that equipment with magical properties.

In both cases, the shared trait represents capability, while class progression determines specialization.

This relationship preserves player freedom while rewarding long-term investment into a chosen discipline.

12.8 Interclass Consistency

Traits serve as the common connection between otherwise unrelated classes.

Because every class references the same underlying framework, progression remains coherent across the entire game.

This allows:

- Different classes to benefit from similar capabilities.
- Equipment to function consistently across professions.
- Shared mechanics to remain reusable.
- Future systems to integrate without introducing redundant progression statistics.

Traits should unify gameplay systems rather than fragment them.

12.9 Design Philosophy

Traits should encourage flexibility without eliminating specialization.

A player should never be prevented from attempting an activity simply because they selected a different class.

However, players who invest heavily into the progression systems designed around that activity should naturally outperform those who do not.

This distinction allows experimentation while preserving meaningful identity.

12.10 Future Expansion

When introducing new mechanics, designers should first determine whether the existing trait framework already provides an appropriate representation of player capability.

Only after exhausting the canonical seven traits should alternative progression systems be considered.

Maintaining a stable trait framework improves balancing, scalability, and long-term maintainability.

12.11 Design Guidelines

When working with traits:

- Treat traits as universal capability measurements.
- Keep calculation logic centralized within KubeJS.
- Recalculate traits whenever relevant player state changes.
- Preserve the canonical seven-trait framework.
- Prefer reusing existing traits over creating new progression statistics.
- Separate capability from specialization.
- Allow classes to determine expression rather than capability.

Traits provide the common language through which Honorable's progression systems communicate, while class progression determines how that capability is ultimately realized by the player.

13 Implementation Guidelines

The Honorable player system framework is intended to separate progression definition from gameplay implementation. Each system should have a clearly defined responsibility and should avoid duplicating functionality that belongs elsewhere.

Maintaining this separation improves maintainability, scalability, and future expansion.

13.1 Puffish Skills Responsibilities

Puffish Skills serves as the structural definition of player progression.

It should be responsible for:

- Class structure
- Skill tree layout
- Node organization
- Active ability acquisition
- Passive acquisition
- Attribute rewards
- GameStage rewards
- Commands
- Progression milestones

Puffish Skills should define progression, not gameplay logic.

13.2 KubeJS Responsibilities

KubeJS serves as the implementation layer for gameplay mechanics.

It should be responsible for:

- Trait calculation
- Temporary modifiers
- Ability behavior
- Derived gameplay calculations
- Equipment interaction
- Event handling
- Runtime checks
- Persistent player data
- Scaling logic

Whenever gameplay behavior becomes sufficiently complex, it should be implemented within KubeJS rather than Puffish Skills.

13.3 GameStage Responsibilities

GameStages represent permanent progression milestones.

They should primarily gate:

- Recipes
- Features
- Structures
- Mechanics
- Quest progression
- Ability access
- Equipment access

GameStages should generally be binary values indicating whether a progression milestone has been achieved.

13.4 Quest Responsibilities

Questing should complement class progression rather than replace it.

Quest rewards may include:

- Items
- Abilities
- Recipes
- Resources
- Exploration unlocks
- GameStages
- Narrative progression

The class tree should establish identity while quests expand the player's opportunities within the world.

13.5 Equipment Responsibilities

Equipment exists as an external progression system.

Equipment should provide:

- Improved efficiency
- Additional utility
- Build diversity
- Temporary specialization
- Enhanced performance

Equipment should amplify player investment rather than replace it.

A highly specialized player should derive greater value from equivalent equipment than a generalized player.

13.6 Separation of Concerns

Each system should own its own domain.

System	Primary Responsibility
Puffish Skills	Progression structure
KubeJS	Gameplay implementation
GameStages	Progression state
Questing	World progression
Equipment	External progression

Whenever multiple systems appear capable of implementing a mechanic, preference should be given to the system whose responsibility most closely aligns with that mechanic.

13.7 Data-Driven Design

Progression should remain data-driven whenever practical.

Skill trees should define:

- What is unlocked.
- When it is unlocked.
- What rewards are granted.

Gameplay systems should determine:

- How those rewards function.
- How they scale.
- How they interact.
- How they are balanced.

This separation minimizes maintenance cost and allows systemic changes without restructuring progression.

13.8 Runtime Flexibility

Because gameplay logic resides primarily within KubeJS, systems should remain capable of responding dynamically to:

- Equipment changes
- Trait recalculation
- Buffs
- Debuffs
- Environmental effects
- Server configuration
- Future mechanics

Progression should remain stable while runtime behavior evolves.

13.9 Future-Proofing

New mechanics should integrate into existing systems whenever possible.

Before introducing additional progression frameworks, designers should ask:

- Can this be represented through traits?
- Can this be implemented through KubeJS?
- Can this be unlocked through GameStages?
- Can this be expressed through existing class progression?

Only after exhausting existing architecture should entirely new systems be introduced.

13.10 Design Principles

When implementing new content:

- Keep progression declarative.
- Keep gameplay procedural.
- Separate unlocks from implementation.
- Reuse existing systems before creating new ones.
- Favor composition over duplication.
- Maintain clear ownership between subsystems.

The long-term maintainability of Honorables depends on preserving these boundaries as the project grows. Systems should cooperate through well-defined interfaces rather than overlapping responsibilities, ensuring that future expansion remains predictable and scalable.

14 Class Design Principles

Every class added to Honorables should contribute a distinct gameplay loop and meaningful role within the broader progression ecosystem.

Classes should not exist merely to provide a different collection of abilities or statistical bonuses. Instead, they should represent a fundamentally different method of interacting with the world.

The purpose of a class is to answer a single question:

“What does this player primarily contribute to the world?”

14.1 Core Gameplay Loop

Every class should define a clear and repeatable gameplay loop.

Examples include:

- Resource acquisition
- Resource production
- Exploration
- Combat mastery
- Magical specialization

The gameplay loop should remain recognizable from Root progression through Mastery.

Subclasses should refine this loop rather than replace it.

14.2 Economic Contribution

Every class should provide value beyond personal progression.

Examples include:

- Producing equipment
- Producing consumables
- Discovering new locations
- Providing combat capability
- Supporting infrastructure
- Enhancing equipment

A healthy multiplayer economy should naturally emerge from interdependent class identities.

14.3 Distinct Identity

No class should exist solely because it uses a different weapon or performs a similar role with minor variation. Instead, classes should differentiate themselves through:

- Progression philosophy
- Gameplay loop
- Economic contribution
- Mechanical identity
- Long-term specialization

Weapon preference alone is insufficient justification for an independent class.

14.4 Foundation Consistency

Every class should follow the standardized Foundation structure.

- One Root node.
- Three parallel branches.
- Seven tiers per branch.

Foundation progression should emphasize experimentation and broad competence rather than specialization.

14.5 Subclass Philosophy

Subclasses exist to refine identity.

They should:

- Expand existing mechanics.
- Increase decision density.
- Introduce specialization.
- Encourage commitment.

Subclasses should never invalidate Foundation progression.

14.6 Mastery Philosophy

Mastery should perfect an established specialization rather than introduce an entirely new one.

Capstones should reinforce:

- Existing mechanics.
- Existing gameplay loops.
- Existing player identity.

Mastery represents refinement through dedication.

14.7 Traits and Classes

Traits provide universal capability.

Classes determine how that capability is expressed.

Two players with identical traits should still perform differently because of differences in:

- Passives
- Perks
- Active abilities
- Augments
- Specialization
- Mastery

Class identity should emerge from progression rather than raw statistics.

14.8 Player Freedom

Classes should encourage specialization without preventing experimentation.

Players should be able to participate in activities outside their specialization.

However, those who invest heavily into a discipline should naturally outperform those who do not.

Identity should arise through investment rather than restriction.

14.9 Long-Term Scalability

When creating new classes, designers should prefer extending existing systems over introducing new ones.

Questions to consider include:

- Does this class provide a unique gameplay loop?
- Does it contribute meaningfully to the economy?
- Does it justify three subclasses?
- Does it interact naturally with the trait framework?
- Does it reinforce rather than duplicate another class?

If the answer to these questions is no, the concept may be better represented as a subclass instead.

14.10 Design Guidelines

Every class should define:

- A Core Gameplay Loop.
- An Economic Contribution.
- Primary Concerned Traits.
- Three Foundation branches.
- Three Subclasses.
- A coherent Mastery path.

The resulting progression should encourage specialization while remaining consistent with the broader Honorables framework.

The goal is not to create as many classes as possible, but to ensure that every class meaningfully expands the ways in which players interact with the world.
